

PYTHON PRACTICE

Three small problems. One stronger Python habit.

A focused challenge pack for beginners who want to practise problem-solving instead of only watching tutorials.

Work independently, use the hints only when needed, then compare your approach with the answer key.

INSIDE

- Three short exercises with clear acceptance checks.
- Three increasingly specific hints for each challenge.
- A complete answer key for self-review or tutoring.

Recommended starting knowledge: variables, lists, functions, and basic if statements.

Practice note: write your own solution. Do not submit the answer key as schoolwork.

CHALLENGE 01

Top words

Write `top_words(text, limit=3)`. Return (word, count) pairs sorted by count descending and then word alphabetically. Ignore punctuation, treat uppercase and lowercase as the same, and raise `ValueError` when limit is negative.

Starter code

```
def top_words(text, limit=3):  
    pass
```

Check your work

```
assert top_words('Cat dog cat bird dog cat', 2) == [('cat', 3), ('dog', 2)]  
assert top_words('zebra ant zebra ant', 2) == [('ant', 2), ('zebra', 2)]  
try:  
    top_words('hello', -1)  
except ValueError:  
    pass  
else:  
    raise AssertionError('negative limit should fail')
```

Hint ladder

1. Normalize the text before counting.
2. A dictionary can store each word's count.
3. Sort with a key containing (-count, word).

Core idea: separate the problem into cleaning, counting, and ordering.

CHALLENGE 02

Average positive numbers

Write `average_positive(numbers)`. Return the average of values strictly greater than zero. Ignore zero and negative values. Raise `ValueError` when there are no positive values.

Starter code

```
def average_positive(numbers):  
    pass
```

Check your work

```
assert average_positive([2, 4, 6]) == 4  
assert average_positive([-4, 0, 2, 4]) == 3  
try:  
    average_positive([-1, 0])  
except ValueError:  
    pass  
else:  
    raise AssertionError('no positive values should fail')
```

Hint ladder

1. The word strictly means $\text{value} > 0$.
2. Collect the accepted values before dividing.
3. Validate that the collection is not empty before calling `sum` and `len`.

Core idea: validate the edge case before doing the calculation.

CHALLENGE 03

Binary search

Write `binary_search(values, target)`. The input list is sorted. Return the index of target, or -1 when it is absent. Do not use `list.index`.

Starter code

```
def binary_search(values, target):  
    pass
```

Check your work

```
assert binary_search([1, 3, 5, 7, 9], 5) == 2  
assert binary_search([1, 3, 5, 7, 9], 9) == 4  
assert binary_search([1, 3, 5, 7, 9], 4) == -1
```

Hint ladder

1. Track the inclusive low and high boundaries.
2. Compare the middle value and discard the half that cannot contain the target.
3. Continue while `low <= high`; move a boundary past middle after each comparison.

Core idea: one comparison can remove half of the remaining search space.

Compare, do not copy

Use the reference solutions after you have made a serious attempt. Read the code line by line and explain why each edge case is handled.

1. Top words

```
def top_words(text, limit=3):
    if limit < 0:
        raise ValueError('limit cannot be negative')
    cleaned = ''.join(char for char in text.lower()
                       if char.isalnum() or char.isspace())

    counts = {}
    for word in cleaned.split():
        counts[word] = counts.get(word, 0) + 1
    return sorted(counts.items(),
                  key=lambda item: (-item[1], item[0]))[:limit]
```

2. Average positive numbers

```
def average_positive(numbers):
    positives = [number for number in numbers if number > 0]
    if not positives:
        raise ValueError('no positive numbers')
    return sum(positives) / len(positives)
```

3. Binary search

```
def binary_search(values, target):
    low, high = 0, len(values) - 1
    while low <= high:
        middle = (low + high) // 2
        if values[middle] == target:
            return middle
        if values[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return -1
```

Next step: change one requirement, add one test, and make the solution handle it honestly.